

Deadline. The homework is due **11:59pm, Mon 04/13, 2026**. You must submit your solutions (in pdf format generated by LaTeX) via GradeScope.

Late Policy. You have up to four grace days (calendar day) for the entire quarter. You don't lose any point by using grace days. If you decide to use grace days, please specify how many grace days you would like to use and the reason at the beginning of your submission.

Academic Integrity. You can discuss with your classmates or use online resources. In particular,

- **Collaboration Policy.** You can get help from the instructor, TAs, or your classmates, but only after you have thought about the problems on your own. **If you discussed with anyone, you need to acknowledge them in your report.**
- **Other sources.** It is OK to get inspiration (but not solutions) from books or online resources, again after you have carefully thought about the problems on your own. **If you use any reference or webpage, or discussed with anyone, you must cite it fully and completely in your report.**
- **AI Tools.** If you use ChatGPT or similar AI tools, you have to attach the full conversation to make it clear what help you obtained from them.

However, you **CANNOT**:

- share your report/code with anyone else,
- read other's report/code,
- copy anything from other source,
- get help from others without acknowledging them in your report, or use any relevant resources (e.g., online article/AI tools such as ChatGPT) without citing them.

Otherwise it will be considered cheating. We reserve the right to deduct points for using such material beyond reason. **In summary, you must write up your solution independently, and type your answers word by word on your own: close the book/notes/online resources when you are writing your final answers. Add all the citations and collaborators' names to the end.**

Write-up. Please use LaTeX to prepare your solutions. For all problems, **please explain how you get the answer** instead of directly giving the final answer, except for some special cases (will be specified in the problem). For all algorithm design problems, please describe using **natural language**. You can present pseudocode if you think that helps to illustrate your idea, but please do not only give some code without explanation. **In grading, we will reward not only correctness but also clarity and simplicity. To avoid losing points for hard-to-understand solutions, you should make sure your solutions are either self-explanatory or contains good explanations.** See some more details here <https://www.cs.ucr.edu/~yihans/teaching/214/S26/assignments/>

Bonus questions. Bonus questions are not required, and the points are not counted in the basic 100 points of the class. Most of the bonus questions are just slightly harder than the regular questions, but some can be extremely hard. In a nutshell, most of them are worthy trying, but don't be frustrated if you cannot solve *all* of them :) We will not provide too many hints/solutions for bonus problems, but feel free to discuss them with your classmates.

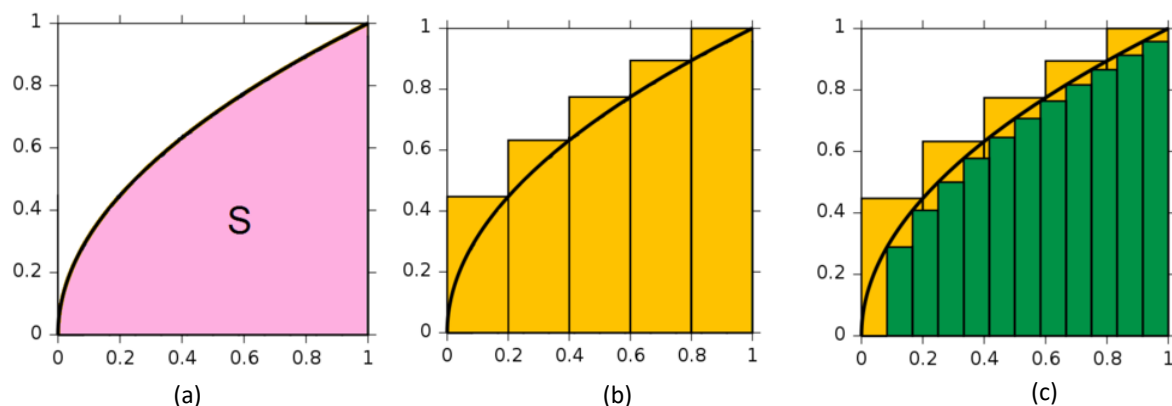


Figure 1: Compute integrals. Source: Wikipedia

1 (10pts + bonus) Programming: Compute the Integral

In this problem, we are going to use a parallel algorithm to compute **integrals**:

$$\int_a^b f(x) dx$$

In some cases, it's hard (or impossible) to compute the close form of the integral. However, we can use numerical solutions. One of the solutions is to sum up the discrete values to estimate the actual value (see Figure 1(b) and (c)).

The idea is, you split the area evenly into small “stripes”, say n of them. Each of them is a rectangle, and we just need to sum up the area of all these rectangles. The height of each rectangle is the corresponding function value, and the width (conceptually dx) is $(b - a)/n$.

You can easily come up with the following pseudocode to compute the integral of function f .

```
double integral(function f, size_t n, double low, double high)
{
    double ans = 0.0;
    for (int i=0; i<n; i++) {
        double x = low + (high-low)*i/n;
        double dx = (high-low)/n;
        ans += f(x) * dx;
    }

    return ans;
}
```

Obviously, the larger the number of rectangles n is, the more accurate the answer is. However, when n is large, this can also be very expensive. So in this problem, let's try to come up with a parallel implementation for this problem. Throughout this problem, we will use the following integral as an example of test case:

$$\int_1^2 \sin(x) \ln(x) dx \tag{1}$$

The answer is about 0.3740279123255. Again, you can find the template file of assignment 1 by using this invitation link: <https://classroom.gitub.com/a/IZ-Lfpi>

(2pts) First Attempt

1. (1pt) What happens if we change the for-loop directly to a parallel-for loop? Will it still give you the correct answer? Briefly explain why.
2. (1pt) What is the running time of the sequential algorithm? Please test on the course server and report the running time. You should expect your final solution to be faster than this baseline.

(8pts) A More Efficient Version

3. (5pts) Design a parallel algorithm that can compute the integral in parallel. You need to show a parallel implementation.

In the repository of homework 1, you will find two files: `integral.cpp` and `integral.h`. Your task is to implement the function `integral` in `integral.h`. `integral.cpp` provides the interface for testing your implementation. It now uses Equation 1 as an example with $n = 10^9$. You can use this function to help you debug. Your output should be about 0.3740279123255. In the testing code (`integral.cpp`), we will run this algorithm for four times and calculate the average running time of the last three rounds.

In our final test, we will only test your `integral.h` file. This means that any changes in your other files (`integral.cpp`, `Makefile`, or any others) won't be kept. In our final test, we will use a similar function with different upper- and lower-bound for the integral, also with $n = 10^9$. Although the test case can be different from the example, you can expect they have similar behavior. You need to write a short report to describe your algorithm and implementation. **In your report please specify your GitHub id.**

Your task is to use whatever way you want to improve the performance of this code. Parallelism will be one of them, but you can also add more to make it perform better!

Note that to make sure your algorithm gives the same answer as specified, please do not change the value of n . In particular, your final answer should match what the sequential algorithm gives. In other words, you can use whatever way to want to speed up the original computation, but please do not change the computation itself.

Your final grade for this part is determined by both the performance of your code and your report. If your algorithm finishes within 3s, you can get full marks for the running time part.

Running time: 5pts	
$\geq 4s$	$\leq 2pts$
2-4s	3pts
1.2-2s	4pts
$\leq 1.2s$	5pts
$\leq 1s$	5pts + bonus + candies (see below)
Report: 3pts	

Note that to get the running time lower than 1s, you probably need to add some optimizations to the naive implementation.

Bonus Questions

4. (up to 1pt bonus and 2 candies) If your algorithm finishes within 1.2s, you can get up to 1pt bonus and up to 2 candies. To get the bonus points, your report needs to describe the optimizations you used to improve performance.
5. (up to 1pt bonus and 2 candies) A nice performance study of using AI to solve the task, see below.

What to Include in the Report? And Some Hints!

In general, your report will record what you discover and how you solve the problem in the project. But if you are new to writing a “report”, here is an outline you can consider. **Three points in this assignment will be assigned based on the clarity of your report.**

- What is the algorithm? Please restate the algorithm based on your own understanding.
- Based on your understanding of the algorithm, what do you think are the most critical to improve the performance?
- Based on your answer for the questions above, what optimizations did you try? Why you think they would work?
- How did you come up with the optimizations? Is it from what you learned in class, reading papers, finding materials online, or based on prior knowledge you have?
- How does each of your optimization work? Please use comparisons of experimental results to show this.
- Do the optimizations work as you expected or not? If not, what other efforts do you try to understand this?

Here are some other interesting things that you can try:

- If you find out that getting performance to be 1.2s or below 1s is not easy to achieve, you can (or you probably will anyway) ask some AI to look into this problem. Given this task, what optimization did your AI give? What prompt did you use to get these optimizations?
- What is the motivation for the AI to use these optimizations? Do they work better than yours?
- For each of the optimization did by you and the AI, what is the performance improvement? Which of them are the most effective? Which of them actually does not help?
- What happens if you change different prompt? Does the AI always give you the same answer? When seeing all the different versions provided by the AI, can you get a better solution by combining them?

If you did a non-trivial study on how you can use the AI for performance engineering this task, we may consider giving bonus points and candies based on the report quality. Note that the bonus points is not given if you just “use” the AI, but we will require you to do an in-depth study of the effectiveness of AI, with clear analysis, discussions, and experimental evaluations.